

Exercise sheet 03 - Computational Statistical Physics

Iason Kazazis

Physics MSc/ 25-935-701 22/3/2026

Exercise 1. Cluster algorithms

Task 1: Connection probability in the Ising model

We start from the q -state Potts model, where the Hamiltonian is

$$H = -J \sum_{\langle i,j \rangle} \delta_{\sigma_i, \sigma_j},$$

and the Boltzmann factor for a single bond is

$$e^{\beta J \delta_{\sigma_i, \sigma_j}}.$$

In the Fortuin–Kasteleyn representation, this can be rewritten as

$$e^{\beta J \delta_{\sigma_i, \sigma_j}} = (1 - p) + p \delta_{\sigma_i, \sigma_j},$$

with

$$p = 1 - e^{-\beta J}.$$

Now consider the Ising model, which corresponds to the special case $q = 2$ with spins

$$s_i \in \{+1, -1\}.$$

The Hamiltonian is

$$H = -J \sum_{\langle i,j \rangle} s_i s_j.$$

We relate the Kronecker delta to the Ising spins:

$$\delta_{s_i, s_j} = \frac{1 + s_i s_j}{2}.$$

Substituting into the Potts Hamiltonian:

$$H = -J \sum_{\langle i,j \rangle} \frac{1 + s_i s_j}{2} = -\frac{J}{2} \sum_{\langle i,j \rangle} 1 - \frac{J}{2} \sum_{\langle i,j \rangle} s_i s_j.$$

The first term is a constant and does not affect probabilities, so the effective interaction is

$$H_{\text{eff}} = -\frac{J}{2} \sum_{\langle i,j \rangle} s_i s_j.$$

Thus, compared to the standard Ising form, the coupling is effectively

$$J_{\text{Ising}} = \frac{J_{\text{Potts}}}{2}.$$

Substituting into the Potts bond probability,

$$p = 1 - e^{-\beta J_{\text{Potts}}} = 1 - e^{-2\beta J_{\text{Ising}}}.$$

Therefore, the bond (connection) probability in the Ising model is

$$p = 1 - e^{-2\beta J},$$

for neighboring spins in the same state.

Task 2.

Task 2(a): Wolff algorithm

For the Ising model with spins $s_i = \pm 1$, the bond-addition probability in the Wolff algorithm is

$$p_{\text{add}} = 1 - e^{-2\beta J}.$$

We implemented the Wolff cluster algorithm on a two-dimensional square lattice with periodic boundary conditions. At each update step, a random lattice site is chosen as the seed of a cluster. Nearest neighbors with the same spin orientation are added to the cluster with probability p_{add} , and this procedure is repeated until all cluster boundaries have been checked once. Finally, all spins belonging to the cluster are flipped simultaneously.

In addition to the spin configuration, we keep track of the total magnetisation

$$M = \sum_i s_i,$$

which allows us to evaluate the magnetisation per spin

$$m = \frac{M}{N}.$$

Task 2(b): Comparison with Metropolis

To validate the implementation, we compared the Wolff algorithm with the Metropolis algorithm from Exercise 01. We measured the magnetisation, magnetic susceptibility, and Binder cumulant in a temperature window around the critical temperature

$$T_c \approx 2.269185.$$

For the results shown below, we used a lattice of size $L = 32$.

The measured runtimes were

$$t_{\text{Metro}} = 362.02 \text{ s}, \quad t_{\text{Wolff}} = 31.66 \text{ s}.$$

Therefore, for this simulation setup the Wolff algorithm was approximately

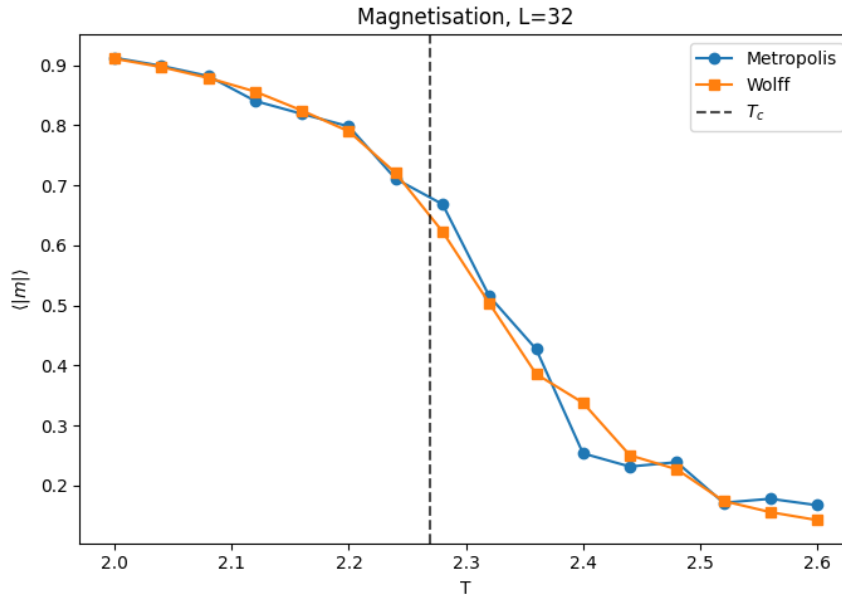
$$\frac{t_{\text{Metro}}}{t_{\text{Wolff}}} \approx 11.4$$

times faster than the Metropolis algorithm. This is physically reasonable, since cluster updates strongly reduce critical slowing down near T_c .

The magnetisation was computed as the finite-size quantity

$$\langle |m| \rangle.$$

The corresponding results are shown in Fig. 1. The two algorithms are in good agreement over the full temperature range. As expected, the magnetisation is large below T_c and decreases rapidly in the critical region.



Σχήμα 1: Magnetisation $\langle |m| \rangle$ for the Metropolis and Wolff algorithms at lattice size $L = 32$. The dashed line marks the critical temperature T_c .

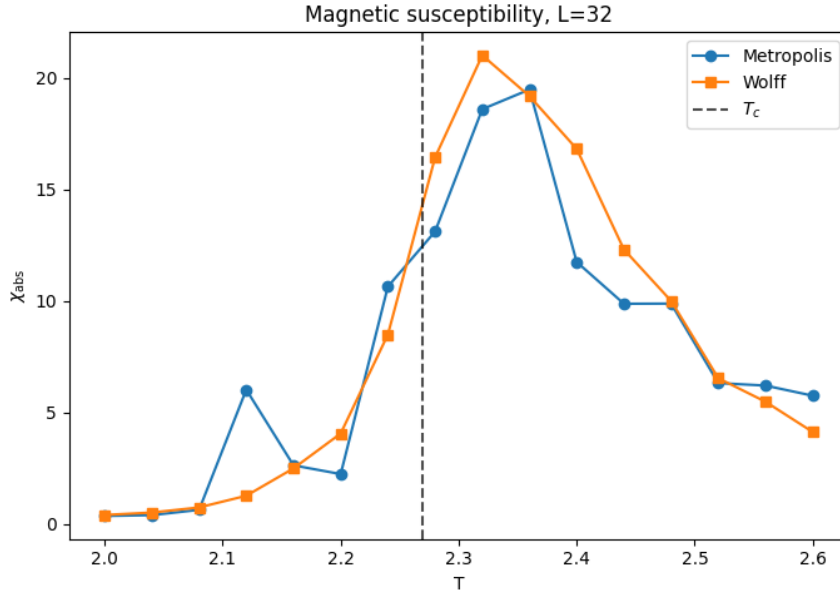
For the magnetic susceptibility, the lecture notes use the fluctuation–dissipation definition

$$\chi = \frac{N}{T} (\langle m^2 \rangle - \langle m \rangle^2),$$

with $k_B = 1$. However, in a finite system the Wolff algorithm can change the sign of the total magnetisation very efficiently, which may drive $\langle m \rangle$ close to zero even below T_c . For this reason, and consistently with the finite-size treatment already used in Exercise 01, we use the corrected estimator

$$\chi_{\text{abs}} = \frac{N}{T} (\langle m^2 \rangle - \langle |m| \rangle^2).$$

The corresponding susceptibility is shown in Fig. 2. We observe a clear peak in the critical region for both algorithms, and the agreement is now qualitatively consistent.



Σχήμα 2: Corrected magnetic susceptibility χ_{abs} for the Metropolis and Wolff algorithms at $L = 32$. The dashed line marks the critical temperature T_c .

For completeness, we also keep the original susceptibility plot in Fig. 3. The difference between the two figures highlights the importance of using a finite-size estimator that is robust against sign changes of the magnetisation in cluster algorithms.

The Binder cumulant was computed as

$$U_L = 1 - \frac{\langle m^4 \rangle}{3\langle m^2 \rangle^2}.$$

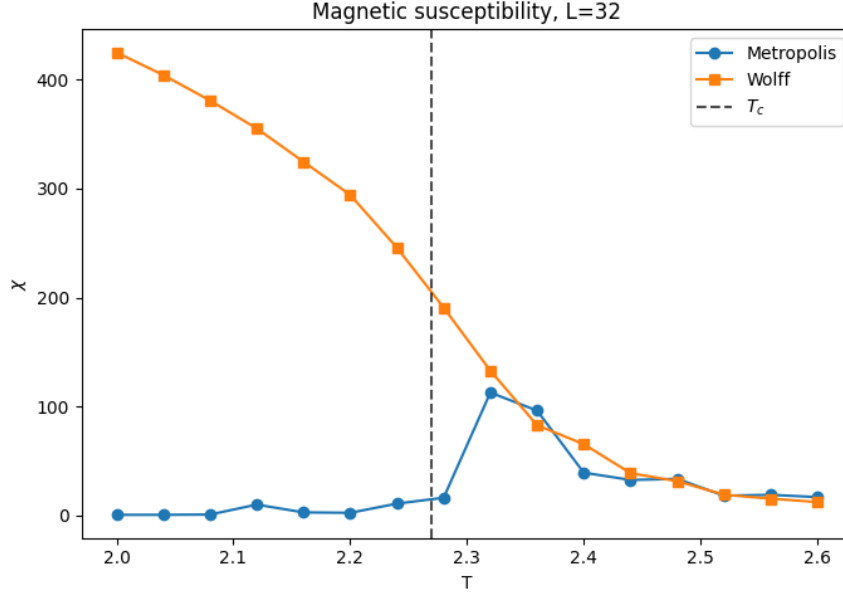
The results are shown in Fig. 4. The behaviour is consistent with the expected finite-size crossover: the Binder cumulant remains close to the ordered-phase value below T_c and decreases above the transition.

Finally, the average cluster size in the Wolff simulation is shown in Fig. 5. As expected, the cluster size increases significantly in the vicinity of the critical temperature, which explains the improved efficiency of the cluster update close to criticality.

Overall, the implementation of the Wolff algorithm is successful. The thermodynamic observables show the expected behavior around the critical temperature, and the comparison with Metropolis confirms that the Wolff algorithm is substantially more efficient. In particular, the runtime reduction by more than one order of magnitude is consistent with the expected suppression of critical slowing down in cluster algorithms.

Task 3.

In this task we compare the performance of the Metropolis and Wolff algorithms by measuring the runtime and the linear autocorrelation time τ using the binning method. Since the Wolff algorithm can change the sign of the magnetisation very efficiently, we estimate the autocorrelation time from the *energy per spin* rather than from the signed magnetisation. This gives a more stable estimate of τ for both algorithms.



Σχήμα 3: Original susceptibility plot before the finite-size correction.

The Monte Carlo speed is defined as

$$MC_{\text{speed}} = \frac{\text{number of steps}}{\text{runtime}} \frac{1}{\tau}.$$

Here, the Metropolis step unit is one sweep, while for the Wolff algorithm one step corresponds to one cluster update.

Task 3(a): fixed lattice size, varying temperature. We first consider a fixed system size $L = 16$ and the three temperatures

$$T_c - 1, \quad T_c, \quad T_c + 1,$$

with

$$T_c \approx 2.269185.$$

The numerical results are shown in Table 1.

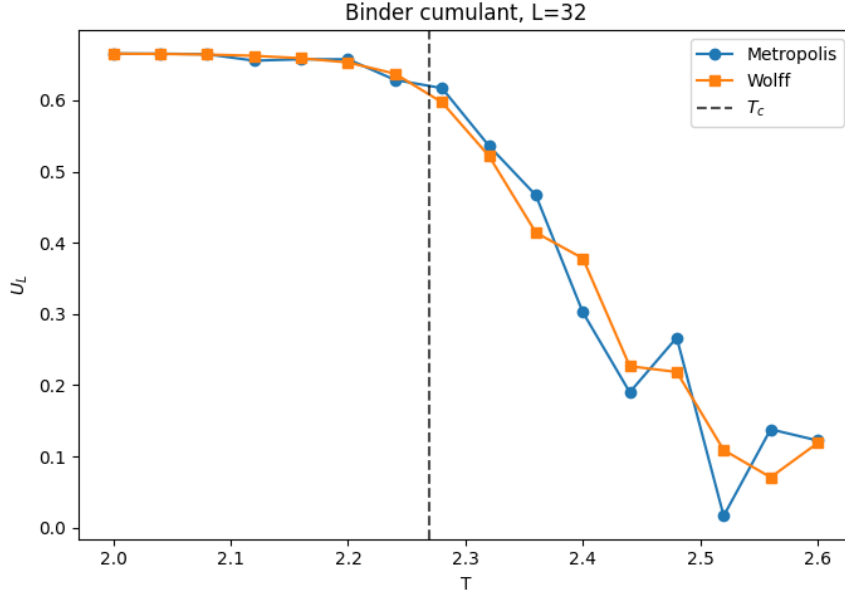
Πίνακας 1: Task 3(a): runtime, autocorrelation time, and Monte Carlo speed for fixed $L = 16$ and temperatures $T_c - 1$, T_c , and $T_c + 1$.

Algorithm	T	Runtime [s]	Steps/s	τ_{bin}	MC_{speed}	Avg. cluster size
Metropolis	1.269185	23.104227	1418.268630	0.974580	1455.260892	–
Wolff	1.269185	12.714954	2577.123032	0.084486	30503.661254	253.885498
Metropolis	2.269185	23.561617	1390.736459	15.454520	89.988976	–
Wolff	2.269185	7.862043	4167.873392	2.961322	1407.436842	139.205994
Metropolis	3.269185	24.745725	1324.188294	1.225436	1080.585407	–
Wolff	3.269185	0.938648	34909.786179	24.267130	1438.562599	7.787628

The Metropolis results show the expected critical slowing down very clearly: the autocorrelation time is small away from the critical point and becomes much larger at T_c . In particular,

$$\tau_{\text{Metro}}(T_c) \approx 15.45,$$

which is significantly larger than its values at $T_c - 1$ and $T_c + 1$. For the Wolff algorithm, the autocorrelation



Σχήμα 4: Binder cumulant for the Metropolis and Wolff algorithms at $L = 32$.

time remains much smaller at the critical point,

$$\tau_{\text{Wolff}}(T_c) \approx 2.96,$$

showing that the cluster update reduces critical slowing down substantially. This is also reflected in the Monte Carlo speed: at T_c the Wolff algorithm is considerably more efficient than Metropolis.

At the same time, one should note that the raw values of τ are measured in different update units for the two algorithms (sweeps for Metropolis and cluster steps for Wolff), so the most meaningful direct comparison is given by MC_{speed} . The results confirm that the Wolff algorithm produces decorrelated samples more efficiently in the critical region.

To extract τ , we used the binning curves shown in Fig. 6. In both cases the curves increase with the binning level and then start to flatten. The Metropolis curve develops a clearer plateau, while the Wolff curve also shows an approximately flat region before the last points. The slight rise at the largest binning levels is attributed to reduced statistics, since only a small number of bins remains there.

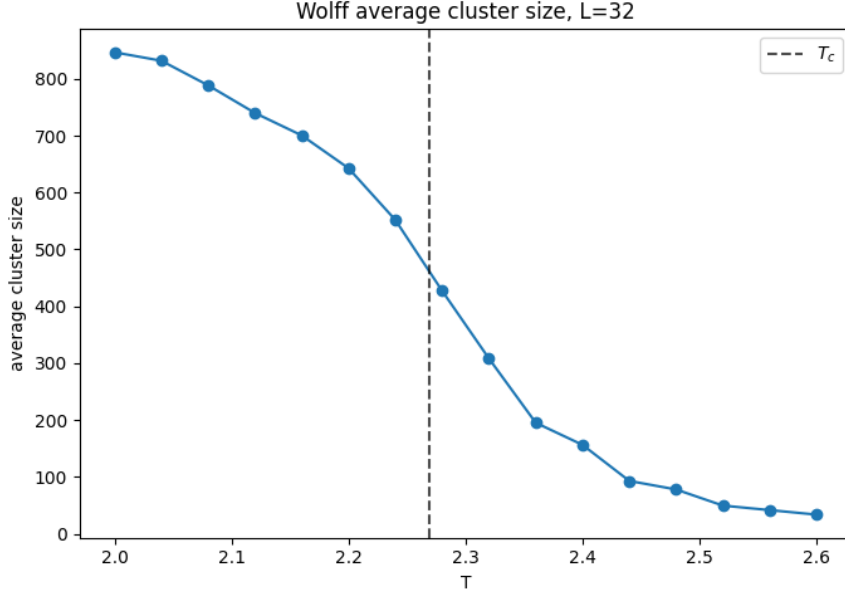
Task 3(b): fixed temperature, varying lattice size. We next fix the temperature to the critical value $T = T_c$ and vary the lattice size,

$$L = 8, 16, 24, 32.$$

The results are shown in Table 2.

Πίνακας 2: Task 3(b): runtime, autocorrelation time, and Monte Carlo speed at fixed $T = T_c$ and varying lattice size.

Algorithm	L	Runtime [s]	Steps/s	τ_{bin}	MC_{speed}	Plateau level	Avg. cluster size
Metropolis	8	3.046665	5377.683155	7.928802	678.246587	9	—
Wolff	8	1.390214	11785.235052	1.460789	8067.721410	9	41.353333
Metropolis	16	11.615697	1410.505056	20.079470	70.246129	9	—
Wolff	16	4.036871	4058.587761	1.945431	2086.216155	5	139.966248
Metropolis	24	25.823049	634.471947	17.423295	36.415152	8	—
Wolff	24	7.780789	2105.698941	3.430598	613.799363	8	283.791931
Metropolis	32	46.173291	354.837175	36.950106	9.603144	9	—
Wolff	32	13.395937	1223.057411	4.097350	298.499617	9	470.434509



Σχήμα 5: Average Wolff cluster size as a function of temperature. The cluster size increases in the critical region.

The Metropolis algorithm shows a clear degradation of performance as the system size increases. The autocorrelation time grows strongly with L , while the Monte Carlo speed decreases rapidly:

$$MC_{\text{speed}}^{\text{Metro}} : 678.25 \rightarrow 9.60 \quad \text{from } L = 8 \text{ to } L = 32.$$

This is the characteristic signature of critical slowing down.

The Wolff algorithm behaves much better. Its autocorrelation time remains much smaller for all lattice sizes, and the Monte Carlo speed stays significantly higher than in the Metropolis case. For example, at $L = 32$ we find

$$\tau_{\text{Metro}} \approx 36.95, \quad \tau_{\text{Wolff}} \approx 4.10,$$

and

$$MC_{\text{speed}}^{\text{Metro}} \approx 9.60, \quad MC_{\text{speed}}^{\text{Wolff}} \approx 298.50.$$

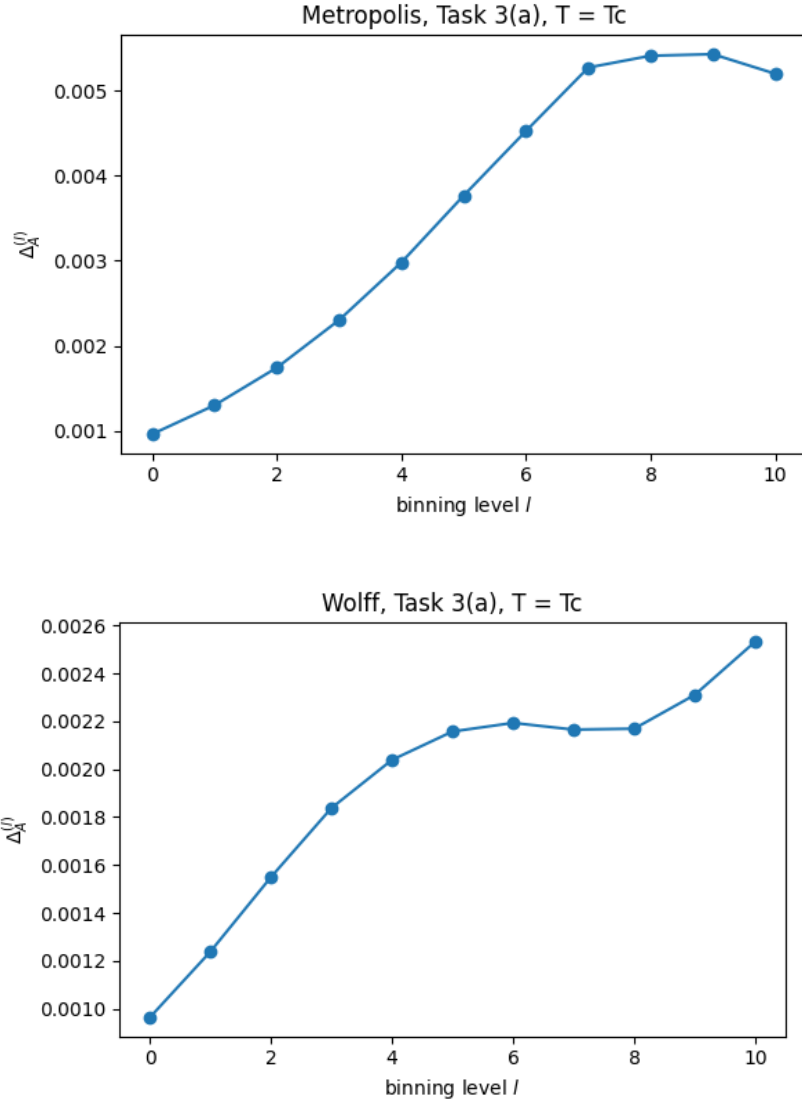
Thus, even for the largest lattice considered here, the Wolff algorithm remains far more efficient.

Another useful quantity is the average cluster size in the Wolff simulation. As expected, it increases strongly with the lattice size:

$$\langle s_{\text{cluster}} \rangle \approx 41.35, 139.97, 283.79, 470.43 \quad \text{for } L = 8, 16, 24, 32.$$

This reflects the fact that near criticality the physically relevant degrees of freedom are correlated on increasingly large length scales, and cluster updates exploit this structure much more efficiently than local single-spin updates.

Conclusion. The results of Task 3 confirm the expected reduction of critical slowing down in the Wolff algorithm. At fixed lattice size, the autocorrelation time is significantly smaller near the critical point than in the Metropolis algorithm. At fixed temperature $T = T_c$, the degradation with increasing system size is also much weaker for Wolff. The corresponding Monte Carlo speeds show that cluster updates produce effectively independent configurations much more efficiently than local Metropolis updates, especially in the critical regime.



Σχήμα 6: Binning curves $\Delta_A^{(l)}$ at $T = T_c$ for Metropolis (top) and Wolff (bottom), used to estimate the autocorrelation time.

Task 4.

As an additional hardware-specific reference, we included the timing of the Exercise 01 reference implementation in the tables of Task 3. The purpose of this comparison is not to replace the autocorrelation analysis of Task 3, but to provide a fixed baseline runtime on the same machine.

The Exercise 01 reference implementation was measured using the same thermalization procedure, lattice sizes, temperatures, and number of measured sweeps as in Task 3. Since it is included only as a timing reference, we report its runtime and step rate, while the autocorrelation-based quantities are left blank.

For Task 3(a), at fixed lattice size $L = 16$, the reference implementation is consistently slower than our Metropolis implementation:

$$31.99 \text{ s} \rightarrow 23.10 \text{ s} \quad \text{at } T = T_c - 1,$$

$$33.83 \text{ s} \rightarrow 23.56 \text{ s} \quad \text{at } T = T_c,$$

$$37.26 \text{ s} \rightarrow 24.75 \text{ s} \quad \text{at } T = T_c + 1.$$

Thus, our Metropolis code is approximately

$$1.38, 1.44, 1.51$$

times faster than the Exercise 01 reference code at the three temperatures considered.

For Task 3(b), at fixed temperature $T = T_c$ and varying lattice size, the same trend is observed. The reference code is again slower than our Metropolis implementation:

$$\begin{aligned} 4.28 \text{ s} &\rightarrow 3.05 \text{ s} \quad (L = 8), \\ 16.99 \text{ s} &\rightarrow 11.62 \text{ s} \quad (L = 16), \\ 38.32 \text{ s} &\rightarrow 25.82 \text{ s} \quad (L = 24), \\ 66.18 \text{ s} &\rightarrow 46.17 \text{ s} \quad (L = 32). \end{aligned}$$

Hence, our Metropolis implementation is roughly

$$1.40, 1.46, 1.48, 1.43$$

times faster than the Exercise 01 baseline for $L = 8, 16, 24, 32$, respectively.

At the same time, the Wolff algorithm remains significantly faster than both the Exercise 01 reference and the Metropolis implementation. For example, at $T = T_c$ we obtain

$$t_{\text{ref}} = 33.83 \text{ s}, \quad t_{\text{Metro}} = 23.56 \text{ s}, \quad t_{\text{Wolff}} = 7.86 \text{ s}$$

for $L = 16$, while at fixed $T = T_c$ and $L = 32$ we find

$$t_{\text{ref}} = 66.18 \text{ s}, \quad t_{\text{Metro}} = 46.17 \text{ s}, \quad t_{\text{Wolff}} = 13.40 \text{ s}.$$

This confirms that the Exercise 01 reference code provides a useful baseline for the local-update algorithm, but also that cluster updates are far more efficient in the critical regime.

Overall, the optional reference benchmark supports the conclusions of Task 3. First, our Metropolis implementation is already more efficient than the Exercise 01 baseline on the same hardware. Second, the Wolff algorithm still outperforms both local-update implementations by a large margin near criticality, confirming the substantial reduction of critical slowing down.

Task 5.

Task 5 (optional). In this task we repeated Tasks 2 through 4 for the Swendsen-Wang algorithm and compared its performance with the Metropolis algorithm.

For the Ising model with spins $s_i = \pm 1$, the Swendsen-Wang algorithm uses the same Fortuin-Kasteleyn bond probability as the Wolff algorithm,

$$p_{\text{add}} = 1 - e^{-2\beta J}.$$

At each update step, nearest-neighbour spins with the same sign are connected with probability p_{add} . This decomposes the lattice into clusters, and each cluster is then flipped independently with probability $1/2$.

To validate the implementation, we measured the magnetisation, the magnetic susceptibility, and the Binder cumulant around the critical temperature

$$T_c \approx 2.269185.$$

For the plots shown below we used a lattice of size $L = 32$.

The total runtimes for this simulation were

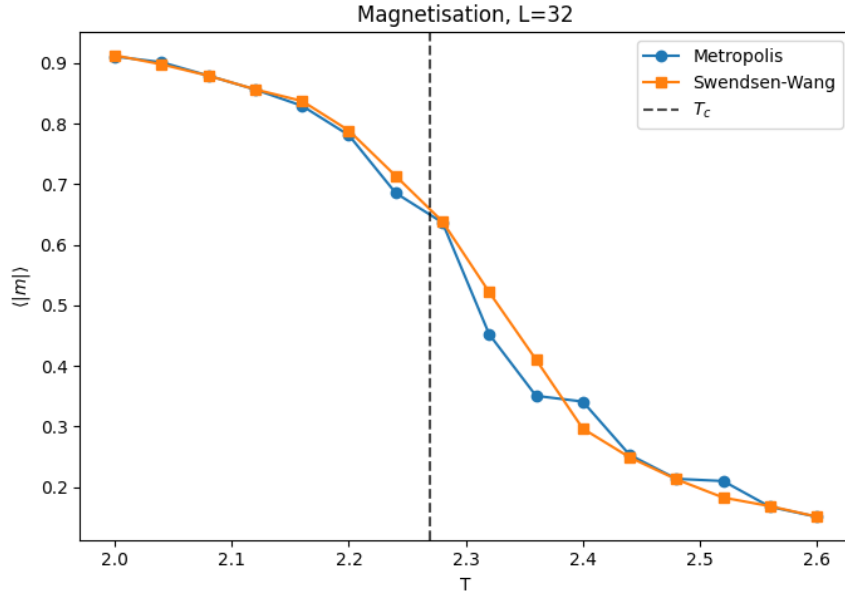
$$t_{\text{Metro}} = 373.53 \text{ s}, \quad t_{\text{SW}} = 29.29 \text{ s}.$$

Hence, in this run the Swendsen–Wang algorithm was approximately

$$\frac{t_{\text{Metro}}}{t_{\text{SW}}} \approx 12.75$$

times faster than the Metropolis algorithm.

The magnetisation results are shown in Fig. 7. The two algorithms are in very good agreement over the full temperature range. As expected, $\langle |m| \rangle$ is large below T_c and decreases rapidly across the critical region.



Σχήμα 7: Magnetisation $\langle |m| \rangle$ for the Metropolis and Swendsen–Wang algorithms at $L = 32$.

For the magnetic susceptibility, the standard fluctuation–dissipation estimator is

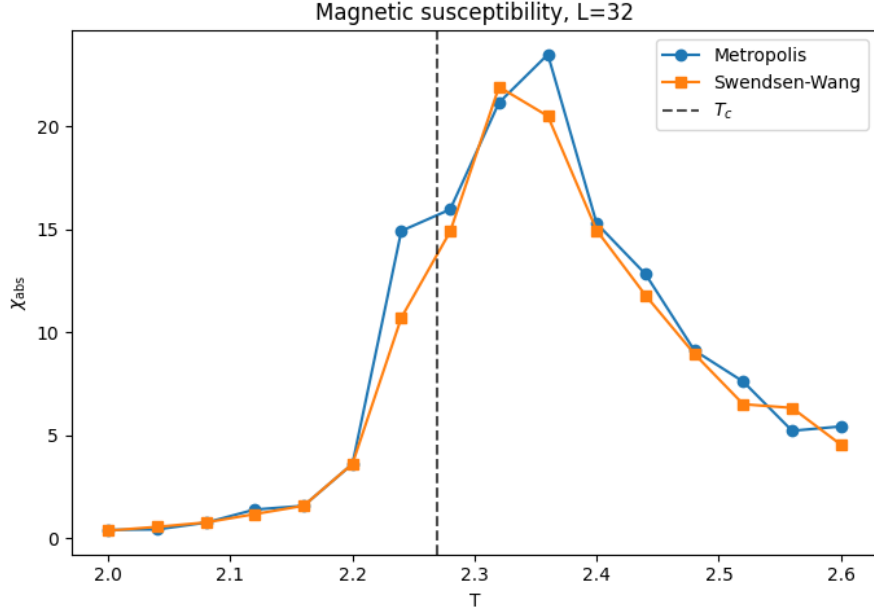
$$\chi = \frac{N}{T} (\langle m^2 \rangle - \langle m \rangle^2).$$

However, in finite systems cluster algorithms can flip the sign of the total magnetisation very efficiently, which makes $\langle m \rangle$ artificially small below T_c . Therefore, for a more robust finite-size comparison we use the corrected estimator

$$\chi_{\text{abs}} = \frac{N}{T} (\langle m^2 \rangle - \langle |m| \rangle^2).$$

The corresponding results are shown in Fig. 8. With this estimator, the Metropolis and Swendsen-Wang curves are in much better agreement and both display a clear peak in the critical region, as expected.

The Binder cumulant is shown in Fig. 9. The behaviour is consistent for both algorithms: the Binder cumulant stays close to the ordered-phase value below T_c and decreases across the critical region.



Σχήμα 8: Corrected magnetic susceptibility χ_{abs} for the Metropolis and Swendsen–Wang algorithms at $L = 32$.

We also measured the average cluster size and the average largest cluster size in the Swendsen-Wang simulation; the results are shown in Fig. 10. The average cluster size remains relatively small and nearly flat because it is dominated by the many small clusters present in the decomposition. In contrast, the largest cluster is strongly temperature dependent and becomes very large below and around the critical temperature, reflecting the growth of long-range correlations.

Task 5(a): fixed lattice size, varying temperature. We next repeated the performance study of Task 3 for fixed lattice size $L = 16$ and temperatures

$$T_c - 1, \quad T_c, \quad T_c + 1.$$

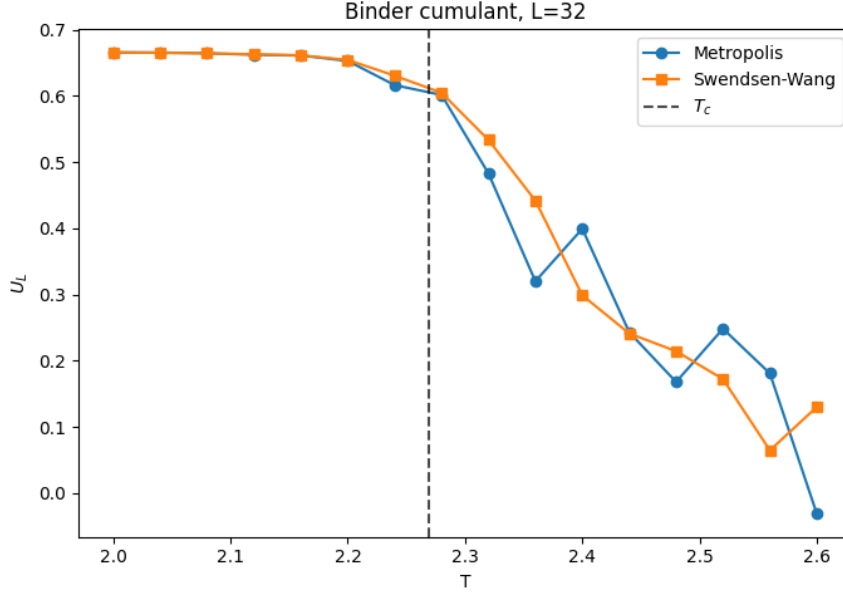
The results are shown in Table 3.

Πίνακας 3: Task 5(a): runtime, autocorrelation time, and Monte Carlo speed for fixed $L = 16$ and temperatures $T_c - 1$, T_c , and $T_c + 1$.

Algorithm	T	Runtime [s]	Steps/s	τ_{bin}	MC_{speed}	Avg. cluster size
Metropolis	1.269185	22.987465	1425.472528	1.161894	1.226852×10^3	—
Swendsen–Wang	1.269185	6.834850	4794.252844	0.004318	1.110250×10^6	159.077703
Metropolis	2.269185	23.372468	1401.991450	28.996637	4.835014×10^1	—
Swendsen–Wang	2.269185	6.355478	5155.866877	0.137579	3.747556×10^4	8.451876
Metropolis	3.269185	24.604967	1331.763610	1.234112	1.079127×10^3	—
Swendsen–Wang	3.269185	6.625564	4945.691917	0.075944	6.512250×10^4	2.547045

The Metropolis autocorrelation time is largest at the critical point, which is the standard signature of critical slowing down. In contrast, the Swendsen-Wang autocorrelation time remains extremely small. Consequently, the Monte Carlo speed of Swendsen-Wang is orders of magnitude larger than that of Metropolis. This shows very clearly that cluster updates drastically improve decorrelation efficiency in the critical region.

To illustrate the extraction of τ , we also show the binning curves at $T = T_c$ in Fig. 11. The Metropolis curve exhibits the expected growth towards a plateau at large binning levels. The Swendsen-Wang curve is



Σχήμα 9: Binder cumulant for the Metropolis and Swendsen–Wang algorithms at $L = 32$.

much flatter, consistent with a very small autocorrelation time. The small oscillations in the last points are attributed to reduced statistics at the largest bin sizes.

Task 5(b): fixed temperature, varying lattice size. We then fixed the temperature to $T = T_c$ and varied the system size,

$$L = 8, 16, 24, 32.$$

The results are shown in Table 4.

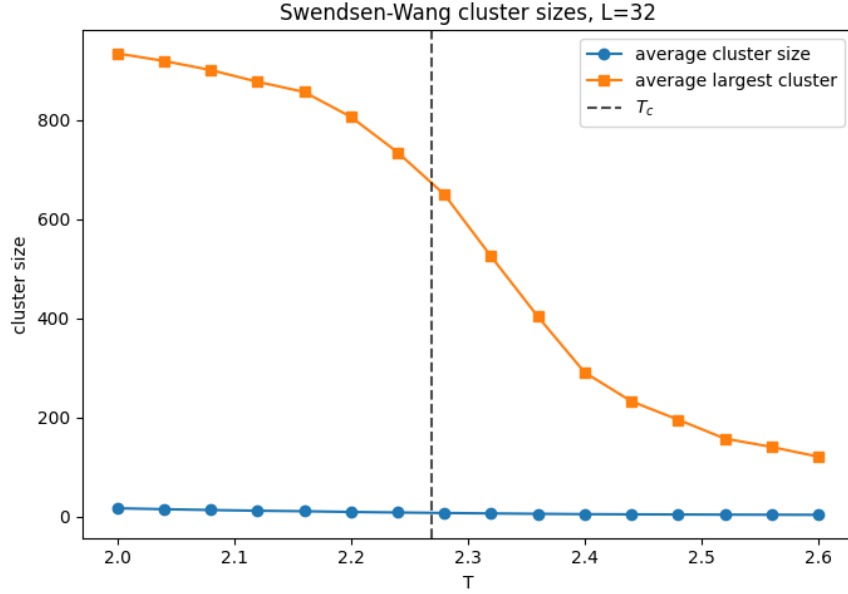
Πίνακας 4: Task 5(b): runtime, autocorrelation time, and Monte Carlo speed at fixed $T = T_c$ and varying lattice size.

Algorithm	L	Runtime [s]	Steps/s	τ_{bin}	MC_{speed}	Plateau level	Avg. cluster size
Metropolis	8	3.112613	5263.744168	5.549392	948.526322	9	–
Swendsen–Wang	8	1.048637	15624.091916	0.127204	1.228270×10^5	8	9.713323
Metropolis	16	11.681525	1402.556625	28.374076	49.430918	9	–
Swendsen–Wang	16	3.358785	4877.954475	≈ 0	–	0	8.520410
Metropolis	24	26.490984	618.474574	39.591416	15.621431	9	–
Swendsen–Wang	24	7.230171	2266.059877	0.025999	8.715887×10^4	8	8.145651
Metropolis	32	49.128430	333.493254	52.122023	6.398318	9	–
Swendsen–Wang	32	11.841237	1383.639247	0.130998	1.056230×10^4	9	8.131331

The Metropolis results again show a clear increase of τ with system size, together with a strong decrease in Monte Carlo speed. This is the expected scaling behavior at criticality. The Swendsen-Wang algorithm performs much better: its runtime grows more slowly and its autocorrelation time remains very small. As a result, its Monte Carlo speed stays much larger than that of Metropolis for all lattice sizes considered.

For $L = 16$, the Swendsen-Wang estimate gave an unresolved value $\tau_{\text{bin}} \approx 0$, which leads to an undefined MC_{speed} in the table. This should not be interpreted as a failure of the algorithm; rather, it indicates that with the present statistics the autocorrelation time is too small to be resolved reliably by the binning estimator. The corresponding qualitative conclusion remains unchanged: the decorrelation in the Swendsen-Wang algorithm is much faster than in Metropolis.

Task 5(c): optional timing reference from Exercise 01. As in Task 4, we also included the Exercise 01 reference implementation as a timing-only baseline. The reference code is consistently slower than



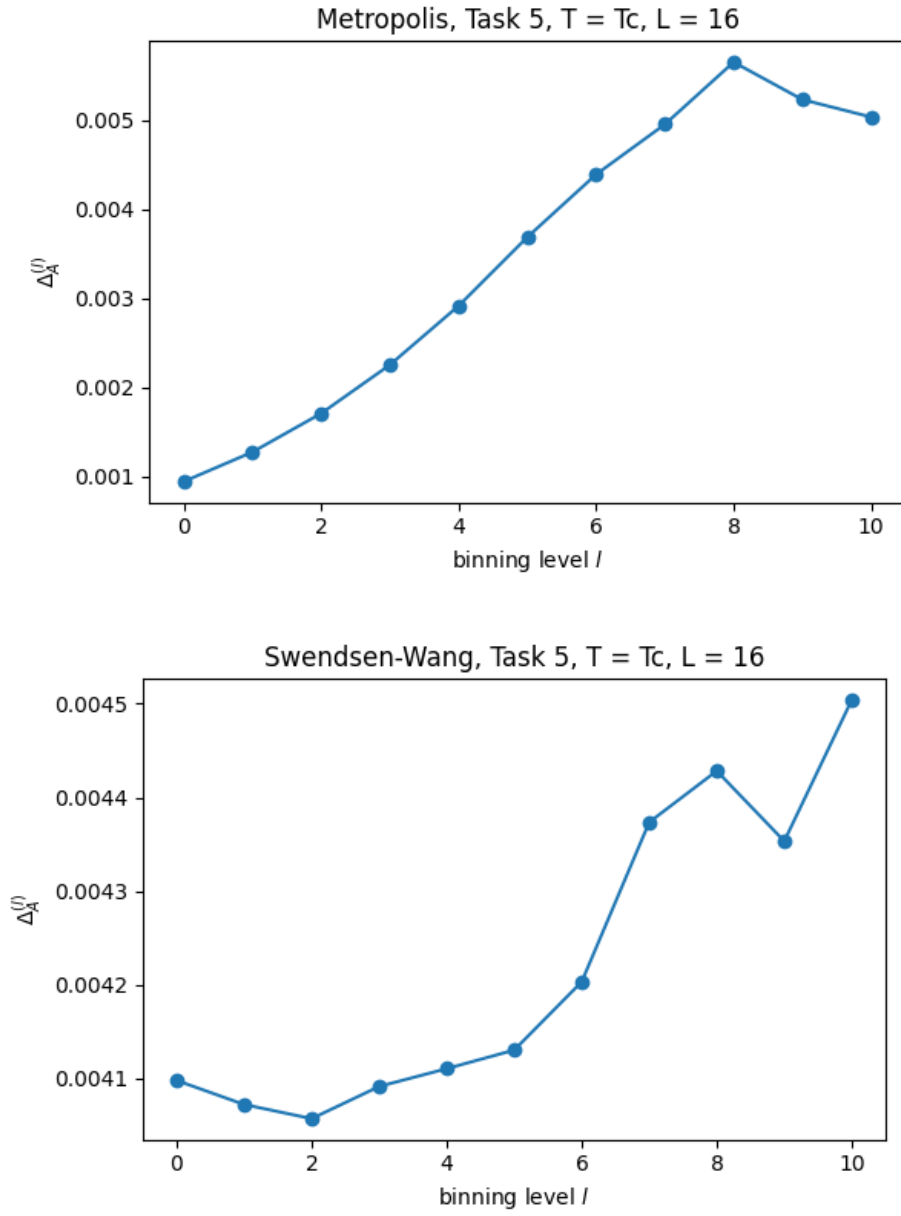
Σχήμα 10: Average cluster size and average largest cluster size for the Swendsen–Wang algorithm at $L = 32$.

our Metropolis implementation and much slower than Swendsen-Wang. This confirms that the speed-up observed here is not merely due to coding details, but is mainly a consequence of the more efficient cluster-update dynamics.

Conclusion. The Swendsen-Wang implementation is successful. The thermodynamic observables are consistent with the Metropolis results, while the runtime and autocorrelation analysis show a dramatic improvement in efficiency. In particular, near the critical temperature the Swendsen-Wang algorithm decorrelates much more rapidly than the local Metropolis update, providing a very strong reduction of critical slowing down.

References

- Course material from the Computational Statistical Physics lectures, including lecture slides and the provided skeleton Python code for the Monte Carlo simulation of the Ising model.



Σχήμα 11: Binning curves $\Delta_A^{(l)}$ at $T = T_c$ for Metropolis (top) and Swendsen–Wang (bottom), used to estimate the autocorrelation time.